How (Neural) Models Learn

Backpropagation, from the chain rule upwards

Tirtharaj Dash

Department of CS & IS
BITS Pilani, Goa Campus, India
Homepage

August 2026

Where we are

A neural model is a parameterised function

$$f_\theta : \mathcal{X} \to \mathcal{Y}, \qquad \theta = (W_1, b_1, W_2, b_2, \ldots)$$

We measure how wrong it is with a loss $L(\theta)$, and then *learn* by changing θ so that L goes down.

Learning is an optimisation problem. Backpropagation supplies the derivatives that the optimisation needs.

The one question

Everything today answers one question:

If I nudge one weight θ_i by a tiny amount, how much does the loss L change?

That number is the partial derivative $\dfrac{\partial L}{\partial \theta_i}$.

A large model can have 10^9 or more weights. We need all of these numbers, cheaply.

Derivative as sensitivity

For a function of one variable,

$$f(x+\epsilon) \approx f(x) + f'(x)\,\epsilon$$

The derivative is the exchange rate between a small change in the input and the change it produces in the output.

Example: $f(x) = x^2$ at $x = 3$, so $f'(3) = 6$.

$$f(3.01) = 9.0601, \qquad f(3) + 6 \times 0.01 = 9.06$$

The error is only 0.0001. Small nudges are all we need.

This is the first-order Taylor expansion, with the $O(\epsilon^2)$ term dropped.

Partial derivatives and the gradient

With several inputs, vary one and hold the rest fixed:

$$\frac{\partial f}{\partial x_i}$$

Collecting them gives the gradient

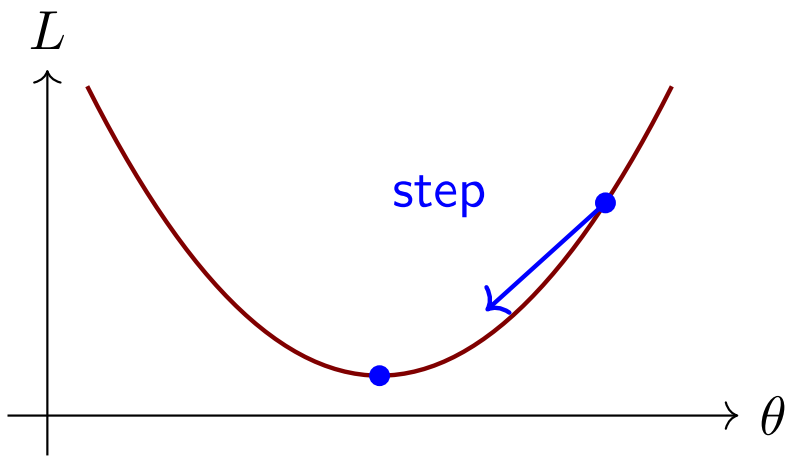
$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n}\right]^{\mathsf{T}}$$

Example: $f(x_1, x_2) = 3x_1^2 + x_1 x_2$ has $\nabla f = [6x_1 + x_2,\ x_1]^{\mathsf{T}}$, which at $(2, 1)$ is $[13, 2]^{\mathsf{T}}$. For equal-sized coordinate perturbations, f is locally 6.5 times more sensitive to x_1 than to x_2.

Gradient descent

Under the Euclidean norm, the gradient points in the direction of steepest increase, so we step against it:

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$



The step size η, the learning rate, is ours to choose. Too small is slow, too large overshoots.

The chain rule: one path

If y depends on u, and u depends on x, then

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Sensitivities multiply along a path.

Example: $y = (3x + 1)^2$ at $x = 2$. Put $u = 3x + 1 = 7$.

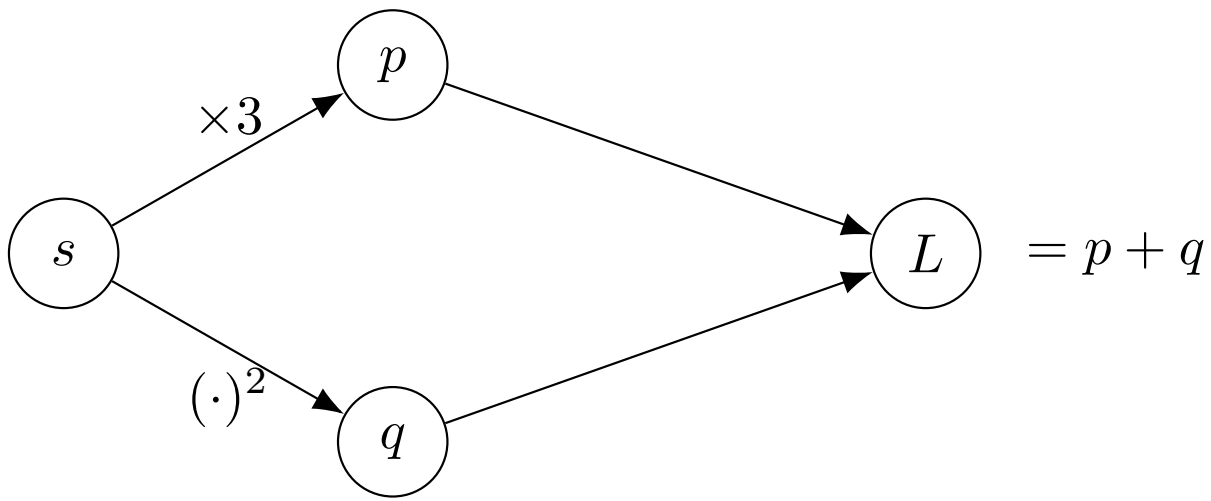
$$\frac{dy}{du} = 2u = 14, \qquad \frac{du}{dx} = 3 \qquad \Longrightarrow \qquad \frac{dy}{dx} = 42$$

Check by expanding: $y = 9x^2 + 6x + 1$, so $dy/dx = 18x + 6 = 42$ at $x = 2$.

The chain rule: several paths

If a quantity reaches the output by more than one route, the contributions add:

$$\frac{dL}{ds} = \sum_{\text{children } c \text{ of } s} \frac{\partial L}{\partial c} \cdot \frac{\partial c}{\partial s}$$



Example: at $s = 2$ we get $p = 6$, $q = 4$, $L = 10$, and

$$\frac{dL}{ds} = 1 \cdot 3 + 1 \cdot 2s = 3 + 4 = 7$$

Check: $L = 3s + s^2$, so $dL/ds = 3 + 2s = 7$.

Why the sum rule matters

Multiply along a path, add across paths.

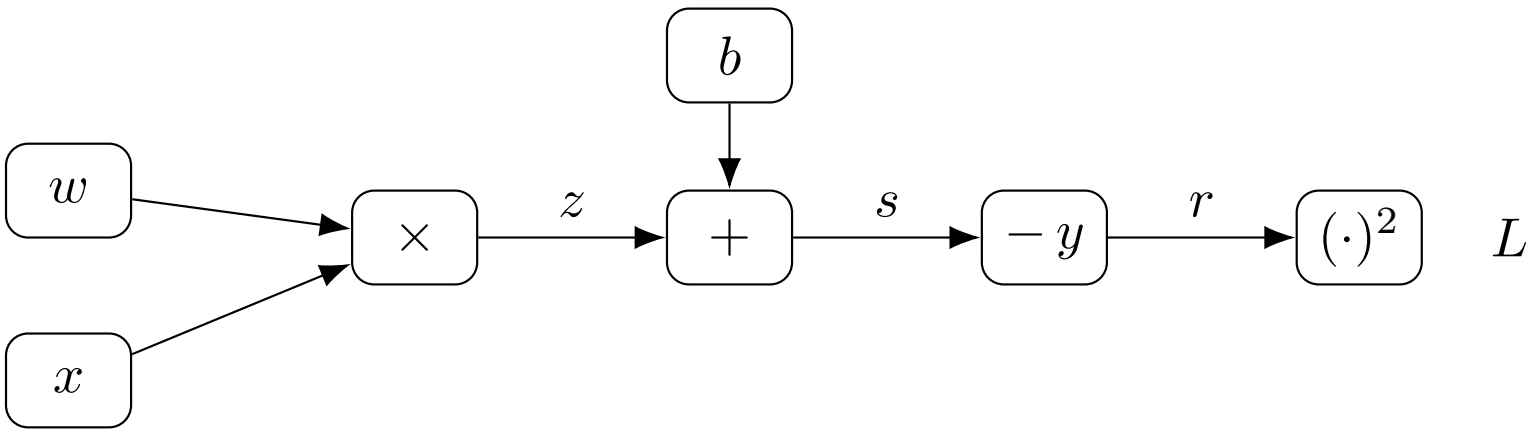
That is all of backpropagation. The rest is bookkeeping: applying these two rules to a large graph in the right order.

The sum rule is what handles shared weights, residual connections, and embeddings that appear at many positions in a sequence. All three turn up later in this course.

Rumelhart, Hinton & Williams, *Nature* 323:533–536, 1986

Computational graphs

Any expression breaks down into primitive operations arranged in a directed acyclic graph. For a single linear neuron with squared error, $L = (wx + b - y)^2$:



Every operation node performs one operation whose derivative we know by hand. Values needed by local derivatives—here, for example, x and the residual r—are retained during the forward pass for use in the backward pass.

The local rule

Each node needs two things:

1. the gradient arriving from above,
2. its own derivative, which it can compute from the values it saw going forward.

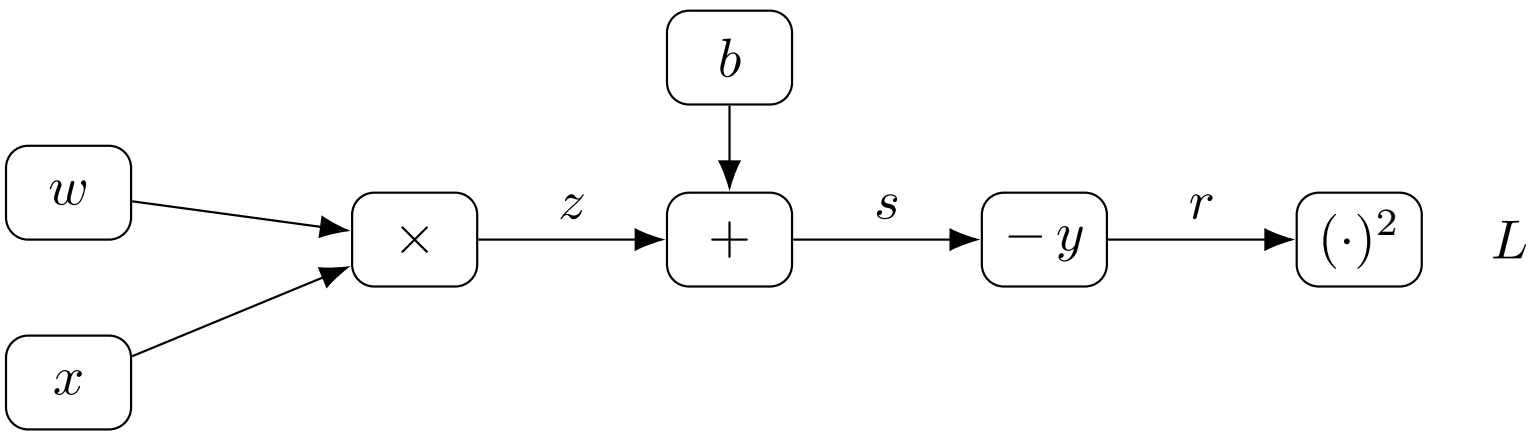
$$\underbrace{\frac{\partial L}{\partial \text{input}}}_{\text{send down}} = \underbrace{\frac{\partial L}{\partial \text{output}}}_{\text{received}} \times \underbrace{\frac{\partial \text{output}}{\partial \text{input}}}_{\text{local}}$$

At a multiplication node $z = wx$, the local derivative with respect to w is just x. No operation node needs to know anything about the rest of the network, and that locality lets the method scale to very large graphs.

AD: Forward mode and reverse mode

Automatic differentiation (AD) applies the chain rule to a program's own operations. Two modes exist, differing only in which end of the graph they start from, and backpropagation is the reverse one.

A *sweep* is one walk over the graph, in topological or reverse-topological order. The forward *pass* carries values, not derivatives.



Let us set $w = 2$, $x = 3$, $b = 1$, $y = 5$, so that $z = 6$, $s = 7$, $r = 2$, $L = 4$.

AD: Forward mode and reverse mode

Forward mode
Seed one input, carry $\dot{v} = \partial v/\partial w$ downstream.

$$\dot{w} = 1, \quad \dot{b} = 0$$
$$\dot{z} = \dot{w}\, x = 3$$
$$\dot{s} = \dot{z} + \dot{b} = 3$$
$$\dot{r} = \dot{s} = 3$$
$$\dot{L} = 2r\, \dot{r} = 12$$

Reverse mode
Seed the output, carry $\bar{v} = \partial L/\partial v$ upstream.

$$\bar{L} = 1$$
$$\bar{r} = 2r = 4$$
$$\bar{s} = \bar{r} = 4$$
$$\bar{b} = \bar{z} = \bar{s} = 4$$
$$\bar{w} = \bar{z}\, x = 12$$

Forward mode gave $\partial L/\partial w$ alone; $\partial L/\partial b$ needs a second derivative sweep. After the forward evaluation, reverse mode obtained both in one reverse sweep.

AD: Forward mode and reverse mode

Why reverse mode is more prevalent.

	Forward mode	**Reverse mode**
Seed at	one input	one output
One sweep gives	all outputs w.r.t. one input	one output w.r.t. all inputs
Derivative sweeps	n, one per parameter	1
Extra memory	low	saved values or recomputation

A scalar loss has one output. After the forward evaluation, reverse mode gets its gradient in one reverse sweep, while forward mode seeded one parameter at a time needs n sweeps. The work of each sweep still scales with the graph.

Packing all n directions into one forward sweep instead gives every $\dot{v}$ an extra dimension of length n and multiplies the arithmetic. Reverse mode avoids this dimension: each $\bar{v}$ has the shape of v.

Baydin et al., *JMLR* 18(153):1–43, 2018

AD: Forward mode and reverse mode

What has to stay in memory

Both modes store numbers; the difference is how long each has to survive. Take a chain $v_1 \to v_2 \to v_3 \to L$, one value per node.

After step	Forward mode holds	Reverse mode holds
1	$v_1, \dot{v}_1$	v_1
2	$v_2, \dot{v}_2$	v_1, v_2
3	$v_3, \dot{v}_3$	v_1, v_2, v_3
4	$L, \dot{L}$	v_1, v_2, v_3, L

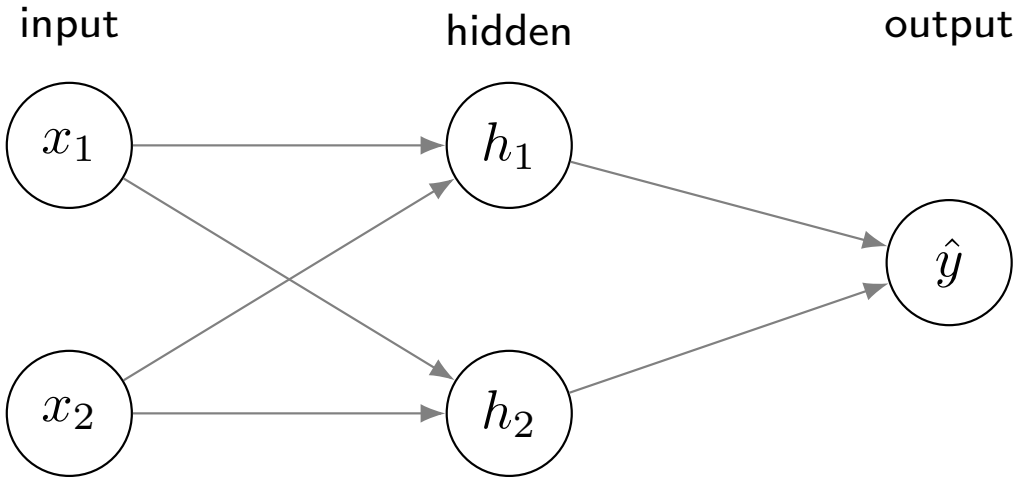
Forward mode releases each pair once the next is computed, so its peak does not grow with depth, and it ends at $\dot{L}$ with no return journey. Reverse mode cannot release v_1 until the reverse sweep returns to it.

Forward mode pays in time: the chain runs again for each of the n parameters.

Baydin et al., *Gradients without Backpropagation*, arXiv:2202.08587, 2022

A small neural network

Two inputs, two hidden units with the logistic function, one output.



$$\boldsymbol{a} = W_1\boldsymbol{x} + \boldsymbol{b}_1, \quad \boldsymbol{h} = \sigma(\boldsymbol{a}), \quad z = W_2\boldsymbol{h} + b_2, \quad \hat{y} = \sigma(z)$$

$$L = \tfrac{1}{2}(\hat{y} - y)^2, \qquad \sigma(t) = \frac{1}{1 + e^{-t}}$$

9 parameters in total: $4 + 2$ in the first layer, $2 + 1$ in the second.

The hidden units h_1, h_2 are the layer the data says nothing about. Backprop decides what they should compute.

Russell & Norvig, *AIMA*, 4th US ed., Ch. 21

We will use these numbers

$$\boldsymbol{x} = \begin{bmatrix} 0.5 \\ -1.0 \end{bmatrix}, \qquad y = 1$$

$$W_1 = \begin{bmatrix} 0.4 & -0.2 \\ 0.1 & 0.6 \end{bmatrix}, \qquad \boldsymbol{b}_1 = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 0.3 & -0.5 \end{bmatrix}, \qquad b_2 = 0.2$$

One fact we will keep using:

$$\sigma'(t) = \sigma(t)\big(1 - \sigma(t)\big)$$

so once we have the activations, the derivative needs very little extra work.

Forward pass (1)

Step F1. Hidden pre-activations:

$$\boldsymbol{a} = W_1 \boldsymbol{x} + \boldsymbol{b}_1 = \begin{bmatrix} 0.4(0.5) + (-0.2)(-1.0) + 0.1 \\ 0.1(0.5) + 0.6(-1.0) - 0.1 \end{bmatrix} = \begin{bmatrix} 0.50 \\ -0.65 \end{bmatrix}$$

Step F2. Hidden activations:

$$\boldsymbol{h} = \sigma(\boldsymbol{a}) = \begin{bmatrix} \sigma(0.50) \\ \sigma(-0.65) \end{bmatrix} \approx \begin{bmatrix} 0.6225 \\ 0.3430 \end{bmatrix}$$

Forward pass (2)

Step F3. Output pre-activation:

$$z = 0.3(0.6225) + (-0.5)(0.3430) + 0.2 = 0.2152$$

Step F4. Prediction and loss:

$$\hat{y} = \sigma(0.2152) = 0.5536$$

$$L = \tfrac{1}{2}(0.5536 - 1)^2 = \tfrac{1}{2}(-0.4464)^2 = 0.0996$$

The target is 1 and we predicted 0.5536. Now we ask how sensitive the loss is to each of the nine parameters.

Backward pass (1)

Step B1. At the prediction:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = -0.4464$$

Negative, so raising $\hat{y}$ would lower the loss.

Step B2. Through the output nonlinearity:

$$\sigma'(z) = \hat{y}(1 - \hat{y}) = (0.5536)(0.4464) = 0.2471$$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \sigma'(z) = -0.4464 \times 0.2471 = -0.1103$$

Note that: $\frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(y - \hat{y})^2$, and either form differentiates to $\hat{y} - y$. The classical delta rule writes the error as $y - \hat{y}$ and adds it in the update instead of subtracting. We will stick to $\frac{1}{2}(\hat{y} - y)^2$.

Backward pass (2)

Step B3. Output layer parameters. Since $z = W_2\boldsymbol{h} + b_2$:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z}\,\boldsymbol{h}^\top = -0.1103 \begin{bmatrix} 0.6225 & 0.3430 \end{bmatrix}$$

$$= \begin{bmatrix} -0.0687 & -0.0378 \end{bmatrix}$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z} = -0.1103$$

Notice the pattern: a weight's gradient is the gradient arriving from above times the activation that weight multiplied on the way forward.

Backward pass (3)

Step B4. Into the hidden activations:

$$\frac{\partial L}{\partial \boldsymbol{h}} = W_2^\top \frac{\partial L}{\partial z} = \begin{bmatrix} 0.3 \\ -0.5 \end{bmatrix} (-0.1103) = \begin{bmatrix} -0.0331 \\ 0.0552 \end{bmatrix}$$

Step B5. Through the hidden nonlinearity, element by element:

$$\sigma'(\boldsymbol{a}) = \boldsymbol{h} \odot (1 - \boldsymbol{h}) = \begin{bmatrix} 0.2350 \\ 0.2253 \end{bmatrix}$$

$$\frac{\partial L}{\partial \boldsymbol{a}} = \frac{\partial L}{\partial \boldsymbol{h}} \odot \sigma'(\boldsymbol{a}) = \begin{bmatrix} -0.00778 \\ 0.01243 \end{bmatrix}$$

Backward pass (4)

Step B6. Hidden layer parameters, by the same pattern as B3:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial \boldsymbol{a}} \boldsymbol{x}^\mathsf{T} = \begin{bmatrix} -0.00778 \\ 0.01243 \end{bmatrix} \begin{bmatrix} 0.5 & -1.0 \end{bmatrix}$$

$$= \begin{bmatrix} -0.00389 & 0.00778 \\ 0.00622 & -0.01243 \end{bmatrix}$$

$$\frac{\partial L}{\partial \boldsymbol{b}_1} = \frac{\partial L}{\partial \boldsymbol{a}} = \begin{bmatrix} -0.00778 \\ 0.01243 \end{bmatrix}$$

All nine partial derivatives, from one backward sweep.

Does it actually help?

Take one gradient step with $\eta = 1.0$, then run the forward pass again:

	before	**after**
$\hat{y}$	0.5536	0.5952
L	0.0996	0.0819

The prediction moved towards the target and the loss fell by about 18%. Repeated updates over many training examples are the basic mechanism by which the network is trained.

Watch the gradient shrink

Track the size of the gradient as it travelled backwards:

$$\left|\frac{\partial L}{\partial z}\right| = 0.110 \qquad \longrightarrow \qquad \left|\frac{\partial L}{\partial a_1}\right| = 0.008$$

A factor of roughly 14 lost along this particular route through one layer.

Here the gradient is multiplied by both $W_{2,1} = 0.3$ and $\sigma'(a_1) = 0.235$, giving the combined factor 0.0705. Thus the shrinkage comes from both the weight and the activation derivative.

Watch the gradient shrink

Across many layers, these weight matrices and activation derivatives multiply. Repeated Jacobian factors with norms below one can make the gradient vanish; factors above one can make it explode. Sigmoid units are especially vulnerable because $\sigma'(t) \leq 0.25$ and tends to zero in saturation.

Glorot & Bengio, *AISTATS*, 2010

Checking our gradients

The derivative is defined as a limit, so we can approximate it directly:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta_i + \epsilon) - L(\theta_i - \epsilon)}{2\epsilon}$$

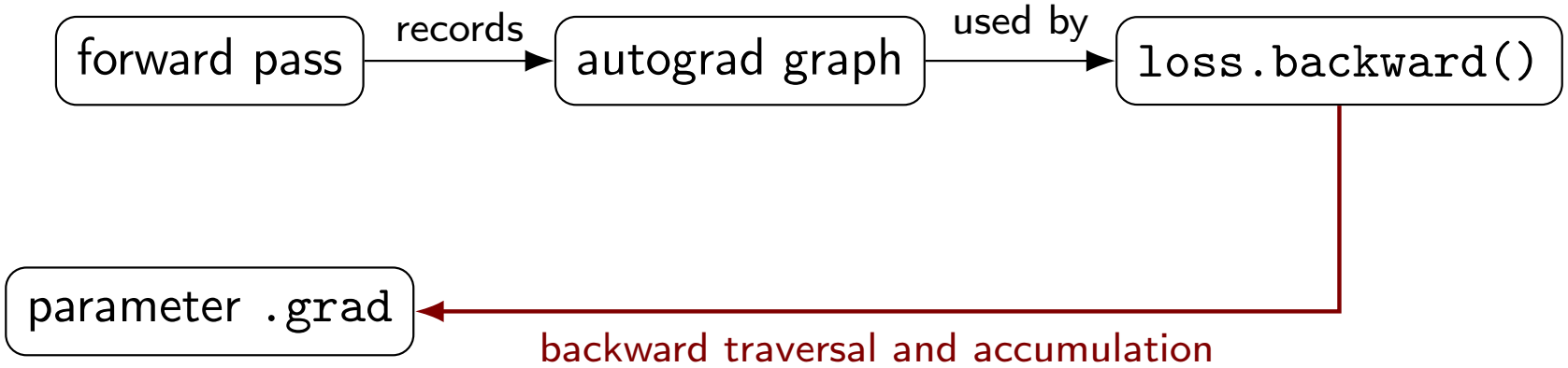
For $W_1[1,1]$ with $\epsilon = 10^{-5}$:

finite difference	-0.00388871
backpropagation	-0.00388871

Agreement to eight decimal places. Far too slow for training, but useful for debugging gradient code that we wrote ourselves.

How autograd does it

PyTorch records operations while the forward pass runs. When `loss.backward()` is called, it traverses this graph in reverse and accumulates gradients in the .grad fields of leaf tensors, such as model parameters.



We will not often write a backward pass by hand. We will often have to reason about one, when training diverges or a gradient comes out zero.

Memory cost of the backward pass

The backward pass uses values computed on the way forward. These are usually saved until backward reaches them, although checkpointing can recompute some values instead to reduce memory.

For a model with N parameters trained with Adam in the same precision, we hold roughly the following numbers of parameter-sized scalars:

parameters	N
gradients	N
Adam moment estimates	$2N$
cached activations	depends on depth and batch size

This is why a model that fits on our GPU for inference may not be trainable on it. We will return to this in the lecture on fine-tuning.

Same rules, bigger graphs

Nothing today assumed anything about the shape of the network. The same two rules apply unchanged to:

- embedding lookups, where gradients accumulate over repeated tokens
- recurrent networks, where one weight is shared across time steps
- causal self-attention, where every position can influence later positions

Later, we will apply these same steps when studying learning in autoregressive language models.

Homework

Work through this by hand first, then in code.

1. Code the worked example from scratch in NumPy and reproduce $\hat{y}$, L and all nine gradients.
2. Rebuild the same network in PyTorch and check `.grad` against your NumPy values.
3. Run a finite-difference check on each of the nine parameters.
4. Train for a few hundred steps and plot L against step number.
5. Keep the sigmoid output unit, but replace the hidden activation σ first with $\tanh$ and then with ReLU. Report $\|\partial L/\partial \boldsymbol{a}\|_2$ in each case. Which hidden activation gives the largest gradient norm for these particular values, and why?

You won't need a GPU for this. The whole notebook will run locally on a CPU within a minute.

Summary

- Learning means descending a loss surface, which needs $\nabla_\theta L$.
- Sensitivities multiply along a path and add across paths.
- A network is a computational graph, and each node needs only a local derivative.
- After the forward evaluation, one reverse sweep gives every parameter's gradient.
- Products of small Jacobian factors can make gradients vanish with depth.

Backpropagation only computes derivatives. Gradient descent is what changes the parameters.

CS F425 Deep Learning, Lectures 3 and 4: Backpropagation in MLPs.